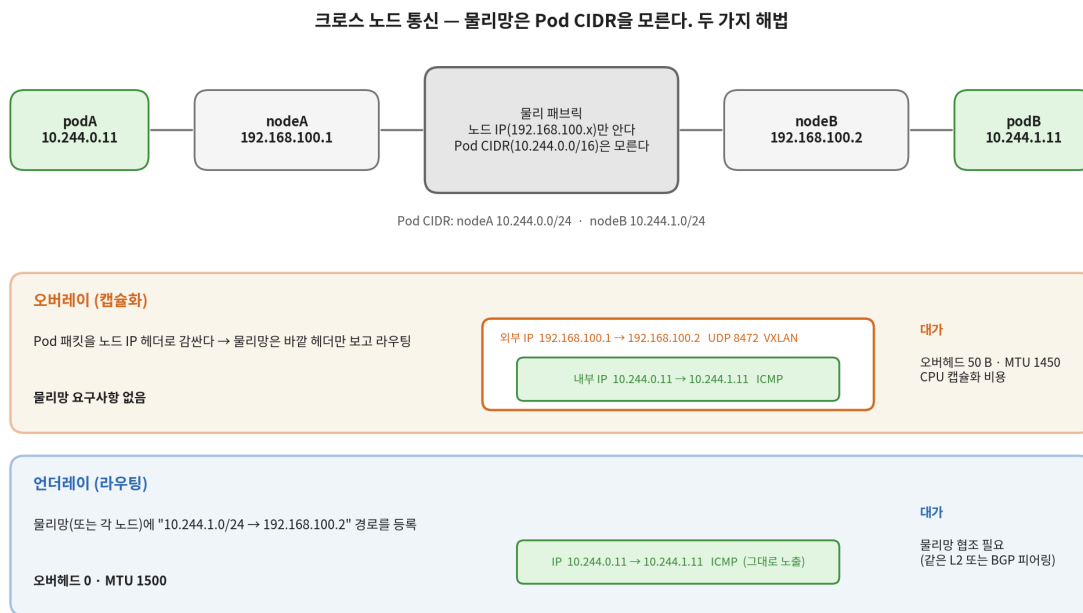


Primary CNI DataPath

1. Cross-Node 통신 문제

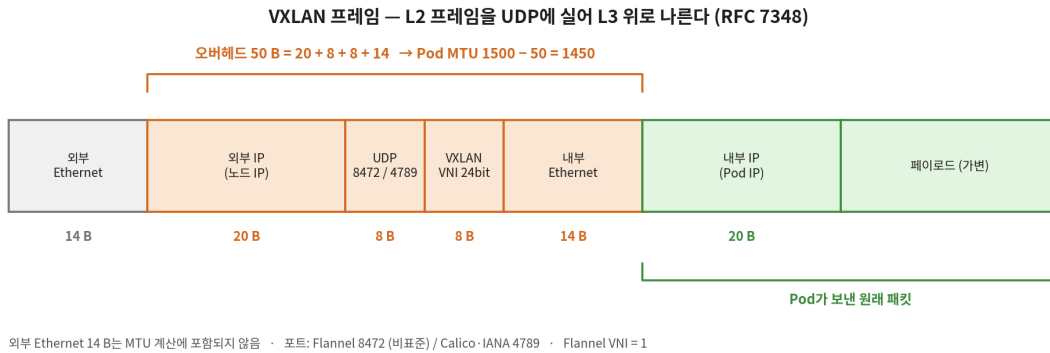


- 물리 네트워크는 노드 IP(192.168.100.x)만 인지하고 Pod CIDR(10.244.0.0/16)은 인지하지 못한다.

접근	방법	대가
오버레이	Pod 패킷을 노드 IP 헤더로 캡슐화	오버헤드, MTU 축소
언더레이	물리망 또는 노드에 Pod CIDR 경로 등록	물리망 협조 필요

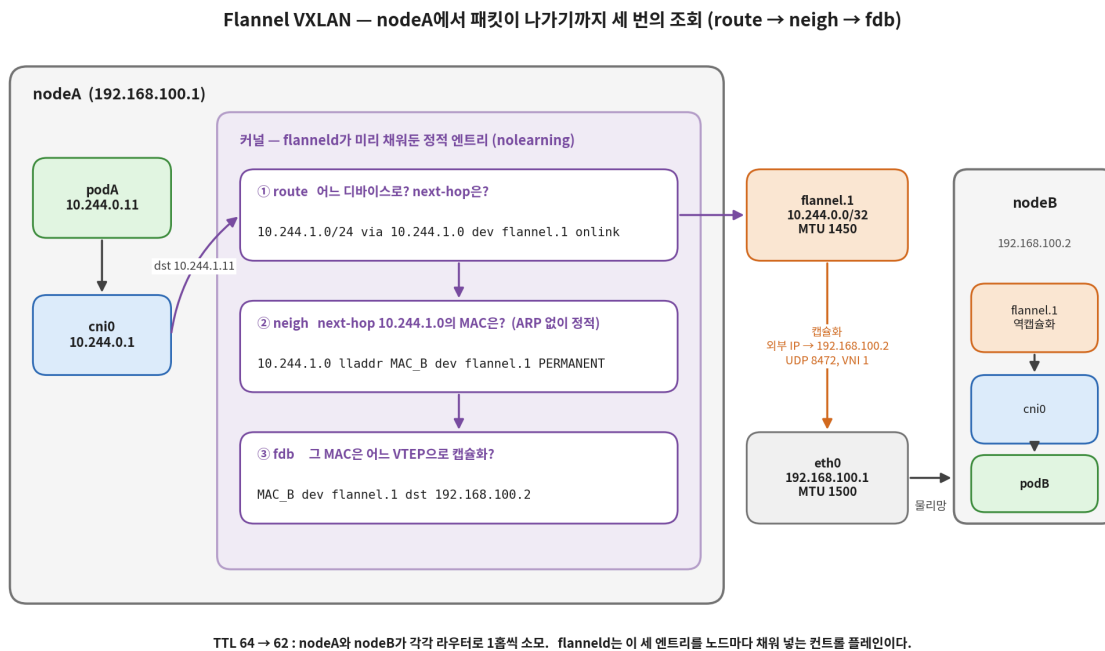
2. Flannel VXLAN

2.1 VXLAN



- L2프레임을 UDP 페이로드에 적재하여 L3 위로 전달 (RFC 7348)
- 오버헤드 50Byte = 외부 IP 20Byte + UDP 8Byte + VXLAN 8Byte + 내부 Ethernet 14Byte
- VXLAN 헤더 8Byte 중 24bit = VNI(논리 네트워크 구분) - Flannel 기본값 1
- 포트
 - IANA 표준 4789 - Calico VXLAN
 - Flannel 8472 - 커널이 표준화 이전에 채택한 포트를 유지
 - 방화벽 규칙 작성 시 구분 필요

2.2 구성



- VXLAN 디바이스 `flannel.1`: `vxlan id 1 dev eth0 dstport 8472 nolearning`
- `nolearning`

- 데이터프레임 MAC 학습 비활성.
- flanneld(컨트롤 플레인)가 모든 엔트리를 명시적으로 기록.
- 멀티캐스트 불필요
- nodeB를 향한 세 엔트리

엔트리	답하는 질문	값
route	어느 디바이스로? next-hop은?	10.244.1.0/24 via 10.244.1.0 dev flannel.1 onlink
neigh	next-hop의 MAC은?	10.244.1.0 lladdr <MAC_B> dev flannel.1 PERMANENT
fdb	그 MAC은 어느 VTEP으로?	<MAC_B> dev flannel.1 dst 192.168.100.2

- next-hop 10.244.1.0 = nodeB Pod CIDR의 네트워크 주소. Flannel이 각 노드 flannel.1에 /32로 부여
- onlink — next-hop이 flannel.1의 서브넷(/32) 밖이므로 필요

2.3 동작 확인 — TTL

```
$ ip netns exec podA ping -c 3 10.244.1.11
64 bytes from 10.244.1.11: icmp_seq=1 ttl=62 time=0.664 ms
```

구성	TTL	경유 라우터
같은 노드, 브리지형	64	0 — L2 스위칭
같은 노드, 라우팅형	63	1 — 호스트
크로스 노드	62	2 — nodeA, nodeB

2.4 캡슐화 캡처

```
$ tcpdump -i eth0 -nn -T vxlan 'udp port 8472'
IP 192.168.100.1.46607 > 192.168.100.2.8472: VXLAN, flags [I] (0x08), vni 1
IP 10.244.1.11 > 10.244.0.11: ICMP echo reply, id 2636, seq 1, length 64
```

- 두 줄이 한 패킷 — 첫 줄 외부 헤더(노드 IP), 둘째 줄 내부 패킷(Pod IP)
- 물리망은 첫 줄만 보고 라우팅
- T vxlan 미지정 시 0TV로 표시 — 포트 8472가 IANA 표준이 아니라 tcpdump가 자동 디코딩하지 않음
- 운영 함정: 물리 NIC에서 Pod IP로 필터하면 아무것도 안 잡힘. udp port 8472 + T vxlan, 또는 flannel.1에서 캡처

2.5 MTU 실측

```
$ cat /sys/class/net/flannel.1/mtu → 1450
$ cat /sys/class/net/eth0/mtu → 1500
```

- 커널이 VXLAN 디바이스 생성 시 부모 MTU - 50 자동 설정
- 프레임 검증 (`ping -s 1000`)

```
캡처 프레임 1092 - 외부 Ethernet 14 = 외부 IP 패킷 1078
내부 IP 패킷 = 1000 + ICMP 8 + IP 20 = 1028
1078 - 1028 = 50 ← VXLAN 오버헤드 정확히 일치
```

- 경계 검증

```
$ ping -M do -s 1423 → 100% loss
$ ping -M do -s 1422 → OK (1422 + 28 = 1450)
```

2.6 host-gw — 언더레이 비교

- VXLAN 라우트 제거 후 노드 IP를 직접 next-hop으로 지정

```
ip route add 10.244.1.0/24 via 192.168.100.2 dev eth0
```

```
$ tcpdump -i eth0 -nn # 패브릭 캡처
IP 10.244.0.11 > 10.244.1.11: ICMP echo request
IP 10.244.1.11 > 10.244.0.11: ICMP echo reply
```

- Pod IP가 물리망에 그대로 노출. 캡슐화 없음, MTU 1500 전부 사용
- 전제: 모든 노드가 같은 L2 세그먼트 — 서로를 직접 next-hop으로 지정 가능해야 함
- 노드가 여러 서브넷에 분산되면 동작 불가

2.7 PMTU 캐시 합정

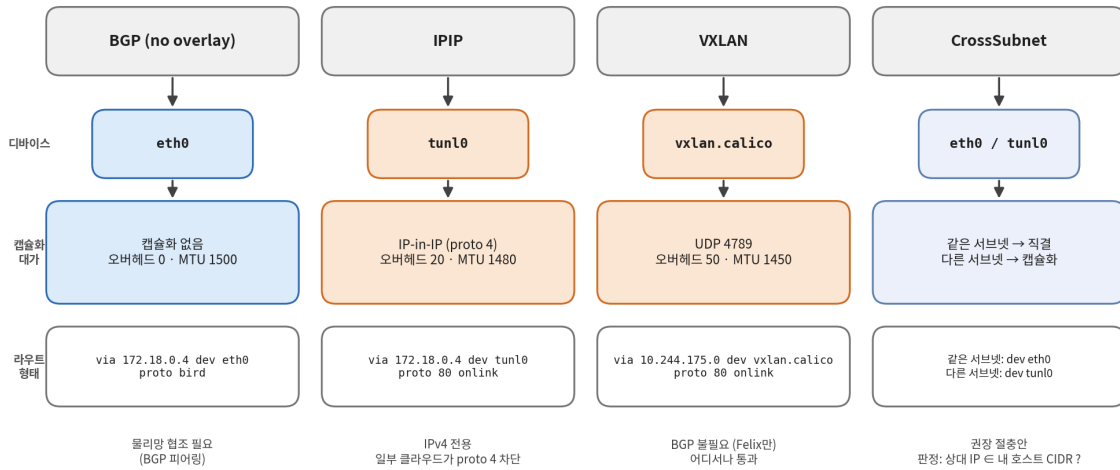
- VXLAN → host-gw 전환 직후 `ping -M do -s 1472` 실패 (1500 가능해야 함)
- 원인: Pod 라우트 캐시의 PMTU

```
$ ip route get 10.244.1.11
10.244.1.11 via 10.244.0.1 dev eth0 src 10.244.0.11
  cache expires 550sec mtu 1450 ← 캐시된 PMTU
```

- `ip route flush cache` 후 정상
- 운영 시사점: MTU 변경·CNI 모드 전환 후에도 기존 Pod는 약 10분간 캐시된 PMTU 유지. 즉시 검증 시 실패로 보임. Pod 재시작 또는 캐시 flush 필요

3. Calico

Calico 노드 간 모드 — 무엇으로 감싸서 어느 디바이스로 내보내는가



모드	캡슐화	디바이스	오버헤드 / MTU	비고
BGP (no overlay)	없음	eth0	0 / 1500	물리망 BGP 피어링 필요
IPIP	IP-in-IP (proto 4)	tunl0	20 / 1480	IPv4 전용. 일부 클라우드(Azure)가 proto 4 차단
VXLAN	UDP 4789	vxlan.calico	50 / 1450	BGP 불필요 — Felix가 직접 라우트 기록
CrossSubnet	목적지에 따라	eth0 / tunl0	혼재	Calico 권장 구성

- IPIP 오버헤드가 VXLAN보다 작은 이유: 내부 Ethernet 헤더 미포함 (외부 IP 20 B만)
- VXLAN 모드 장점: UDP라 어디서나 통과, BIRD 없이 구성 단순

3.1 CrossSubnet — 조건부 캡슐화

- 목적지 노드가 같은 서브넷이면 직결, 다른 서브넷이면 캡슐화
- 판정 기준 (소스 `felix/calc/l3_route_resolver.go`)

```
nowSameSubnet := myNewNodeInfoKnown && myNewV4CIDR.ContainsV4(otherNode.sIPv4)
```

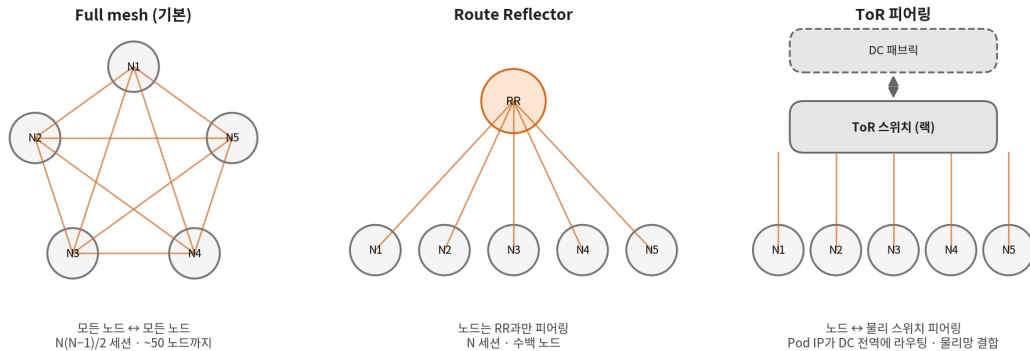
- = "상대 노드 IP가 내 호스트 CIDR 안에 있는가"
- 라우팅 테이블에 두 형태 혼재

```
10.244.175.0/24 via 172.18.0.4 dev eth0 proto 80 onlink ← 같은 서브넷
10.244.99.0/24 via 172.19.0.7 dev tunl0 proto 80 onlink ← 다른 서브넷
```

- **함정:** 노드가 자기 CIDR을 /32 로 보고하면 모든 상대가 cross-subnet 판정 → 사실상 Always로 퇴화. 클라우드 노드 인터페이스 prefix 오탐지 시 발생

3.2 BGP 토폴로지

Calico BGP 토폴로지 — 누가 누구와 피어링하는가



토폴로지	구조	확장성
Full mesh (기본)	모든 노드 ↔ 모든 노드	$N(N-1)/2$ 세션. ~50 노드
Route Reflector	노드 → RR만 피어링	N 세션. 수백 노드
ToR 피어링	노드 → 랙 물리 스위치	DC 전역 라우팅

- ToR 피어링 = Calico의 가장 강력한 구성
 - Pod IP가 DC 전체에서 라우팅 가능
 - 외부 레거시 시스템이 Pod IP로 직접 접근, LB가 Pod를 직접 백엔드로
- 대가: 물리망 결합. 네트워크 팀 협조, Pod CIDR이 물리망 주소 계획에 종속

4. NetworkPolicy

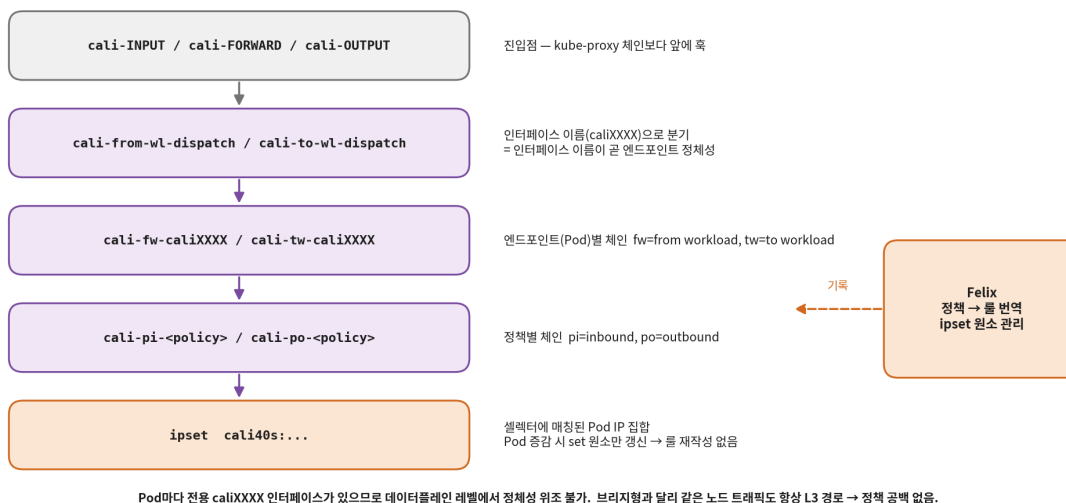
4.1 스펙과 구현의 경계

- NetworkPolicy는 **CNI 스펙 밖**. 집행은 구현체의 자발적 기능

CNI	NetworkPolicy	확장
Flannel 단독	미지원 — 생성해도 무반응, 에러 없음	—
Calico	지원	GlobalNetworkPolicy, order , DNS 정책, 서비스 계정 셀렉터
Cilium	지원	CiliumNetworkPolicy — L7, Identity 기반 (5주차)

4.2 Calcio 집행 구조

Calico NetworkPolicy 집행 — Felix가 정책을 iptables 체인 + ipset으로 번역한다



- Felix가 정책을 iptables(또는 nftables) 체인 + ipset으로 번역
- 체인 계층
 1. cali-INPUT / FORWARD / OUTPUT — 진입점. kube-proxy 체인보다 앞
 2. cali-from-wl-dispatch / cali-to-wl-dispatch — 인터페이스 이름으로 분기
 3. cali-fw-caliXXXX / cali-tw-caliXXXX — 엔드포인트별
 4. cali-pi-<policy> / cali-po-<policy> — 정책별
 5. ipset 매칭 (cali40s:...)
- 인터페이스 이름 = 엔드포인트 정체성. Pod마다 전용 caliXXXX → 데이터플레인 레벨 위조 불가
- 선택자 = ipset. Pod 증감 시 set 원소만 갱신 → 룰 재작성 불필요

4.3 정책 평가 순서

	쿠버네티스 NetworkPolicy	Calico NetworkPolicy
액션	allow만. 다중 정책은 OR	allow / deny / log / pass
순서	없음	order (낮은 값 우선)
범위	네임스페이스	네임스페이스 + Global
선택자	레이블	레이블 + 표현식

- **pass**: GlobalNetworkPolicy에서 조건 충족 시 평가를 네임스페이스 정책으로 위임 → 플랫폼 팀 전역 정책 + 앱 팀 개별 정책 계층화

4.4 브리지형 CNI의 정책 공백

- 브리지형: 같은 노드 Pod 간 트래픽은 L2 스위칭 → **netfilter** 미통과 (1주차 §2.5)
- **br_netfilter** 미적용 시 같은 노드 내 NetworkPolicy 무효

```
sysctl net.bridge.bridge-nf-call-iptables # 1 이어야 함
```

- Calico: 브리지 없음, 항상 L3 경로 → 구조적으로 공백 없음

5. 정리

	Flannel VXLAN	Flannel host-gw	Calico IPIP	Calico VXLAN	Calico BGP
오버헤드	50 B	0	20 B	50 B	0
Pod MTU	1450	1500	1480	1450	1500
물리망 요구	없음	같은 L2	없음	없음	BGP 피어링
로컬 데이터플레인	브리지	브리지	/32 라우팅	/32 라우팅	/32 라우팅
컨트롤 플레인	flanneld	flanneld	BIRD/Felix	Felix only	BIRD
Pod IP 외부 노출	불가	같은 L2 내	불가	불가	DC 전역
NetworkPolicy	X	X	✓	✓	✓